Lesson 13 Teacher's Key: A Hit That Reacts

Scratch - Instructor Reference Only

VIDEO GAME CREATION

What This Lesson Teaches

Sequencing a visual reaction BEFORE finalizing game-state logic using Scratch's "wait" block, and spawning a fully independent visual effect via broadcast - the first lesson where the enemy's removal is deliberately delayed relative to when it was hit.

Level 1 - Full Worked Solution

- The Explosion sprite is a completely separate, self-contained script - it never touches score, lives, or wave variables, and its own "wait 0.2 seconds -> hide" lifecycle needs no help from the Enemy.
- "broadcast [explosion]" fires and moves on immediately - Scratch broadcasts never block the sender, which is exactly why the explosion can be triggered without disturbing the flash-then-wait sequence that follows it.
- "broadcast [enemy hit]" - the one line GameManager actually listens to for score/enemies_remaining/wave-clear - does not move at all. It stays exactly where Lesson 12 left it. The only structural change is adding "wait 0.1 seconds" AFTER it and moving "delete this clone" to after that wait.
- hit_x/hit_y exist because Scratch's broadcast carries no data of its own, and the Enemy is clone-based - "go to [Enemy v]" would always go to the Enemy template sprite's own position, never the specific clone that actually got hit. Setting these two shared variables from the clone's own x position/y position right before broadcasting is the standard Scratch technique for passing information along with a broadcast.
- The flash itself is a genuine costume swap (switch costume to [flash]), not "set [color] effect" - Scratch's color effect shifts hue around the color wheel rather than blending toward white, so it was never capable of producing a true white flash on a colored sprite in the first place, a different-looking version of the same lesson Lesson 2's Recolor and the other seven engines' white-flash fix already taught: a reliable exact color needs a genuine swap, not a blend/effect.

Highlighted lines are new this week; everything else is code you already have.

```
Enemy's touching-Bullet branch (revised):
if <touching Bullet?> then
  start sound [sfx_hit]
  set [hit_x] to (x position)
  set [hit_y] to (y position)
  broadcast [explosion]
  switch costume to [flash]
  broadcast [enemy hit]
```

```
  wait 0.1 seconds
  delete this clone

Explosion sprite's own scripts (NEW - two separate scripts):
when green flag clicked
hide

when I receive [explosion]
show
go to x: (hit_x) y: (hit_y)
wait 0.2 seconds
hide
```



Everything in the top box happens the instant the hit is detected. Only the enemy's own removal - the very last step - waits for the pause to finish.

Why This Structure

- Keeping the explosion as a broadcast-triggered, fully independent script (rather than something the Enemy waits on) means a slow or buggy explosion animation could never accidentally freeze the actual game logic - a good habit even at this small scale.
- This is the first lesson where 'wait' pauses something that MATTERS (delaying a state change) rather than just pacing a visual (Lesson 2's beep-beep) - but notice it's delaying the LEAST important thing happening in this branch (the enemy's own visual removal), not the GameManager broadcast that actually matters. That ordering is the whole lesson.

- hit_x/hit_y are the simplest fix for Scratch's "broadcasts carry no data" limitation - shared variables, set immediately before the broadcast and read immediately after it's received, are the standard pattern any time one sprite needs to hand a value to another through a broadcast.

Level 2 / Level 3 Talking Points

- Ask students to trace, step by step, what happens between a bullet touching an enemy and the enemy actually disappearing now - sound, explosion, flash, broadcast [enemy hit] (GameManager updates score/wave immediately), THEN wait, THEN delete this clone. A good comprehension check for sequencing.
- For strong students: ask what would happen if "broadcast [enemy hit]" were accidentally moved to AFTER "wait 0.1 seconds" instead of staying before it - score and wave-clear would lag 0.1 seconds behind the visuals, which is exactly the mistake this lesson is designed to avoid.
- Ask students what would happen with two enemies hit in quick succession if hit_x/hit_y didn't exist and Explosion just used "go to [Enemy v]" instead - every explosion would appear at the same spot (wherever the Enemy template sprite happens to sit), regardless of which clone was actually hit.

Level 2 - complete code:

```
// Level 2 is verification only - the Level 1 code already produces a
working flash and explosion, so no lines change.
// Checks: play a full round and confirm every single hit flashes and
explodes, with no enemy left stuck mid-flash and no explosion left on
screen;
// confirm score/enemies_remaining/wave-clear/win-check all still happen
at exactly the same instant they did before this lesson - only the enemy's
own final removal is delayed;
// confirm a second bullet hitting an already-flashing enemy does not
double-register a kill.
```

Level 3 - complete code (all three goals combined):

Highlighted lines are the Level 3 additions on top of the Level 1 code.

```
// Enemy's touching-Bullet branch - Level 3 flicker (replaces the single
costume switch;
// Enemy has exactly two costumes, normal and flash in that order, so once
a fresh clone
// starts on normal, three "next costume" blocks simply toggle it back and
forth - this
// only works as a clean toggle because there are exactly two costumes in
that order; if
// Enemy ever gains a third costume, switch back to explicit "switch
costume to [name]"):
if <touching Bullet?> then
  start sound [sfx_hit]
  set [hit_x] to (x position)
  set [hit_y] to (y position)
  broadcast [explosion]
next costume
wait 0.1 seconds
next costume
wait 0.1 seconds
next costume
```

```
  broadcast [enemy hit]
  wait 0.1 seconds
  delete this clone

// Explosion sprite - Level 1 initialization (unchanged, included for
completeness):
when green flag clicked
hide

// Explosion sprite's own script - Level 3 expanding-burst scale (replaces
the static image;
// set size to 100% first, since this is the same Explosion sprite reused
every hit, not a
// fresh clone - without the reset, each explosion would start at whatever
size the last one
// grew to, getting bigger and bigger across the round instead of
resetting each time):
when I receive [explosion]
set size to (100) %
show
go to x: (hit_x) y: (hit_y)
set size to (105) %
wait 0.07 seconds
set size to (110) %
wait 0.07 seconds
set size to (115) %
wait 0.06 seconds
hide
```

Common Student Mistakes

- Moving "broadcast [enemy hit]" to AFTER the wait instead of leaving it where it was, which delays score/wave/win-check for no reason and contradicts this lesson's central rule.
- Forgetting to reset "switch costume to [normal]" somewhere in the enemy's spawn script, causing a freshly spawned enemy to start already showing its flash costume.
- Placing "delete this clone" BEFORE the wait instead of after, which defeats the whole point (the enemy would still vanish instantly, no visible flash).
- Using "go to [Enemy v]" in Explosion's script instead of "go to x: (hit_x) y: (hit_y)" - this compiles and runs with no error, but every explosion appears at the Enemy template sprite's own position instead of wherever the actual clone was hit.
- Using "set [color] effect to 100" for the flash instead of a costume swap - looks like a reasonable guess by analogy with other engines' tint properties, but Scratch's color effect is a hue rotation, not a brightness blend, so it does not produce white.
- (Level 3 flicker) Using "next costume" to toggle the flash after adding a third costume to Enemy for some other purpose - with only two costumes, each clone starting on normal, "next costume" is a reliable toggle; with three or more it starts cycling through all of them instead of flashing, and explicit "switch costume to [name]" blocks become necessary again.